



TIPOS DE DATOS

Data Structures

Carlos Rojas-Sánchez

UNIVERSIDAD DEL MAR

1. Tipos de datos simples
2. Tipos de datos abstractos
3. Clasificación de las estructuras de datos
4. Retroalimentación de operaciones con arreglos y estructuras

Tipos de datos simples

Introducción a los Tipos de Datos Simples

Un **tipo de dato** define:

- El rango de valores que puede almacenar una variable.
- El tamaño en memoria asignado (en bytes).
- Las operaciones permitidas sobre dicho valor.

En lenguaje C, los tipos primitivos/simples son los bloques fundamentales para construir estructuras más complejas.

Enteros (int)

El tipo `int` representa números enteros (sin parte decimal).

- **Tamaño típico:** 4 bytes (32 bits), dependiendo de la arquitectura.
- **Rango con signo:** -2^{31} a $2^{31} - 1$.

```
#include <stdio.h>

int main() {
    int edad = 25;
    int temperatura = -5;
    printf("Edad: %d, Temp: %d\n", edad, temperatura);
    return 0;
}
```

Modificadores de Enteros

Podemos modificar el tamaño y el signo de los enteros:

- `short int` (al menos 16 bits).
- `long int` (al menos 32 bits).
- `unsigned int` (solo valores no negativos, duplica el rango positivo).

```
unsigned int contador = 100u;  
short int nivel = 3;  
long int poblacion = 80000000000L;  
  
printf("Contador positivo: %u\n", contador);
```

Caracteres (char)

El tipo **char** almacena un único carácter o símbolo.

- **Tamaño:** Exactamente 1 byte (8 bits).
- En C, los caracteres internamente son **números enteros** (código ASCII).

```
#include <stdio.h>

int main() {
    char letra = 'A';
    char digito = '7';

    // Imprime el carácter y su valor ASCII
    printf("Letra: %c, ASCII: %d\n", letra, letra);
    return 0;
}
```

Comportamiento numérico de char

Al ser enteros internamente, se pueden realizar operaciones aritméticas con ellos.

```
#include <stdio.h>

int main() {
    char inicial = 'A';
    char siguiente = inicial + 1; // 'B'

    printf("Original: %c\n", inicial);
    printf("Siguiente: %c\n", siguiente);
    return 0;
}
```

Punto Flotante (float)

El tipo `float` representa números reales con decimales (precisión simple).

- **Tamaño típico:** 4 bytes.
- **Precisión:** 6 a 7 dígitos decimales significativos.

```
#include <stdio.h>
```

```
int main() {  
    float pi = 3.14159f;  
    float promedio = 9.5f;  
  
    printf("Pi aprox: %.2f\n", pi);  
    return 0;  
}
```

Doble Precisión (double)

El tipo `double` ofrece mayor precisión para cálculos numéricos exigentes.

- **Tamaño típico:** 8 bytes.
- **Precisión:** 15 a 17 dígitos decimales significativos.

```
#include <stdio.h>
```

```
int main() {  
    double precision_alta = 3.141592653589793;  
    printf("Valor exacto:%.15f\n",precision_alta);  
    return 0;  
}
```

Booleans (bool en C99)

En C original (C89), los booleanos no existían (se usaba 0 para falso y $\neq 0$ para verdadero).

A partir de **C99**, podemos incluir `<stdbool.h>`:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    bool activo = true;
    bool completado = false;

    if (activo) {
        printf("El sistema esta activo (%d)\n", activo);
    }
    return 0;
}
```

El operador sizeof

Permite obtener el tamaño real en memoria (en bytes) de un tipo de dato o variable en el sistema actual.

```
#include <stdio.h>
```

```
int main() {  
    printf("bytes de char:    %zu\n", sizeof(char));  
    printf("bytes de int:     %zu\n", sizeof(int));  
    printf("bytes de float:   %zu\n", sizeof(float));  
    printf("bytes de double: %zu\n", sizeof(double));  
    return 0;  
}
```

Resumen de Tipos Simples

Tipo	Especificador	Tamaño (típico)	Uso Principal
char	%c	1 byte	Caracteres / Texto
int	%d / %i	4 bytes	Enteros
float	%f	4 bytes	Decimales simples
double	%lf	8 bytes	Decimales precisos
bool	%d	1 byte	Lógica booleana

Table 1: Resumen de tipos de datos primitivos en C

Tipos de datos abstractos

¿Qué es un Tipo de Dato Abstracto (TDA)?

Un **Tipo de Dato Abstracto (TDA)** es un modelo matemático para tipos de datos donde el tipo es definido por su **comportamiento** desde el punto de vista del usuario.

Componentes de un TDA

- **Valores:** La colección de datos que forman la estructura.
- **Operaciones:** El conjunto de funciones permitidas sobre los datos (interfaz).

Clave: Se enfatiza *qué* hace la estructura, no *cómo* está implementada internamente.

Abstracción e Encapsulamiento

- **Abstracción:** Oculta la complejidad de implementación tras una interfaz limpia.
- **Encapsulamiento:** Protege la estructura interna agrupando datos y funciones, e impidiendo el acceso directo a los campos.

Ventajas principales

- Modularidad y reutilización de código.
- Facilidad para mantener o cambiar la implementación interna sin romper el código cliente.

Diseño de un TDA en C: Encabezado (.h)

En C, la interfaz pública de un TDA se define habitualmente en un archivo de cabecera (.h).

```
// contador.h
#ifndef CONTADOR_H
#define CONTADOR_H

// Declaración incompleta (Opaque Pointer)
typedef struct Contador Contador;

Contador* contador_crear(int valor_inicial);
void contador_incrementar(Contador* c);
int contador_obtener_valor(const Contador* c);
void contador_destruir(Contador* c);

#endif
```

Implementación del TDA en C (.c)

La definición exacta de la estructura se oculta en el archivo de fuente (.c).

```
// contador.c
#include <stdlib.h>
#include "contador.h"

struct Contador {
    int valor; // Acceso privado al campo
};

Contador* contador_crear(int valor_inicial) {
    Contador* c = malloc(sizeof(Contador));
    if (c != NULL) c->valor = valor_inicial;
    return c;
}
```

Implementación de Funciones del TDA

Completamos el comportamiento del TDA en el archivo `.c`.

```
// contador.c (continuación)

void contador_incrementar(Contador* c) {
    if (c != NULL) {
        c->valor++;
    }
}

int contador_obtener_valor(const Contador* c) {
    return (c != NULL) ? c->valor : 0;
}

void contador_destruir(Contador* c) {
    free(c);
}
```

Uso del TDA desde el Cliente (main.c)

El usuario del TDA interactúa exclusivamente mediante punteros opacos y funciones públicas.

```
#include <stdio.h>
#include "contador.h"

int main() {
    Contador* c = contador_crear(10);

    contador_incrementar(c);
    printf("Valor actual: %d\n", contador_obtener_valor(c));

    // c->valor = 100; // ERROR: El compilador prohíbe
    // acceso directo al miembro

    contador_destruir(c);
    return 0;
}
```

Ejemplos Clásicos de TDA

Pila (Stack): Estructura LIFO (*Last In, First Out*). Operaciones: **push, pop, top.**

Cola (Queue): Estructura FIFO (*First In, First Out*). Operaciones: **enqueue, dequeue, front.**

Lista (List): Secuencia ordenada de elementos. Operaciones: **insertar, eliminar, buscar.**

Conjunto (Set): Colección de elementos únicos sin orden implícito. Operaciones: **agregar, pertenece, union.**

Ejemplo de Especificación: TDA Punto 2D

Interfaz para manipular puntos en un plano cartesiano:

```
// punto.h
typedef struct Punto Punto;

Punto* punto_crear(double x, double y);
double punto_obtener_x(const Punto* p);
double punto_obtener_y(const Punto* p);
double punto_distancia_origen(const Punto* p);
void punto_destruir(Punto* p);
```

Estructura de Datos vs. Tipo de Dato Abstracto

TDA (Abstracto)	Estructura de Datos (Concreto)
Define qué hace el dato.	Define cómo se organizan los datos.
Nivel conceptual / interfaz. Ejemplo: TDA Lista.	Nivel de implementación. Ejemplo: Arreglo dinámico o Lista doblemente enlazada.

Resumen: Buenas Prácticas al Diseñar TDA

1. **Ocultamiento de Información:** Usa punteros opacos (`typedef struct TDA TDA;`) en los archivos `.h`.
2. **Gestión de Memoria:** Proveer explícitamente funciones de creación (`_crear`) y destrucción (`_destruir`).
3. **Validación de Punteros:** Comprobar siempre que los punteros recibidos no sean `NULL`.
4. **Const-correctness:** Utilizar el calificador `const` en parámetros que no modifiquen el estado del TDA.

Clasificación de las estructuras de datos

Las estructuras de datos se pueden clasificar según dos criterios fundamentales de organización e implementación:

Criterios Principales

1. Por su disposición lógica:

- Estructuras **Lineales**
- Estructuras **No Lineales**

2. Por la gestión de memoria:

- Estructuras **Estáticas**
- Estructuras **Dinámicas**

a) Estructuras Lineales

En una estructura **lineal**, los elementos se organizan de forma secuencial, uno detrás de otro.

- Cada elemento (excepto el primero y el último) tiene un único **predecesor** y un único **sucesor**.
- El recorrido de los elementos se realiza en una sola dimensión.

Ejemplos principales:

- Arreglos (*Arrays*)
- Listas enlazadas (*Linked Lists*)
- Pilas (*Stacks*)
- Colas (*Queues*)

Ejemplo de Estructura Lineal en C

Una lista simplemente enlazada organiza los datos en forma de cadena lineal mediante punteros.

```
typedef struct Nodo {  
    int dato;  
    struct Nodo* siguiente; // Apunta al sucesor único  
} Nodo;  
  
// Representación lineal:  
// [Dato 1] -> [Dato 2] -> [Dato 3] -> NULL
```

a) Estructuras No Lineales

En una estructura **no lineal**, un elemento puede estar conectado a múltiples elementos.

- La relación entre los elementos es jerárquica o en red (multidimensional).
- Un elemento puede tener cero, uno o varios predecesores/sucesores.

Ejemplos principales:

- **Árboles:** Estructura jerárquica (raíz, nodos padre/hijo, hojas).
- **Grafos:** Redes compuestas por vértices (nodos) y aristas (relaciones).

Ejemplo de Estructura No Lineal (Árbol)

En un árbol binario, cada nodo puede dividirse en múltiples caminos (izquierdo y derecho).

```
typedef struct NodoArbol {  
    int dato;  
    struct NodoArbol* izquierdo; // Subárbol izquierdo  
    struct NodoArbol* derecho;   // Subárbol derecho  
} NodoArbol;
```

```
/*  
  ( 10 )      <- Raíz  
  /   \  
(5)   (15)   <- Hijos (Múltiples sucesores)  
*/
```

Resumen: Lineales vs. No Lineales

Estructuras Lineales	Estructuras No Lineales
Secuencia secuencial de elementos.	Secuencia jerárquica o multi-direccional.
Un solo nivel de adyacencia.	Múltiples niveles o conexiones libres.
Recorrido sencillo (un solo camino).	Múltiples caminos de recorrido (DFS, BFS).
Uso común: Listas, búsquedas simples.	Uso común: Jerarquías, redes, mapas.

b) Estructuras Estáticas

Las estructuras **estáticas** reservan una cantidad fija de memoria antes de la ejecución o al compilar.

- Su tamaño **no puede modificarse** durante la ejecución del programa.
- La memoria se asigna habitualmente en el *Stack* (pila de ejecución).

Ventajas y Desventajas

- ✓ Acceso rápido y directo mediante índices.
- ✓ Gestión automática de la memoria por el compilador.
- × Ineficiente si no se conoce el tamaño exacto de antemano (desperdicio o desbordamiento).

Ejemplo de Estructura Estática en C

Un arreglo estático asigna un bloque fijo e inmutable en el *stack*.

```
#include <stdio.h>

#define TAMANO 5

int main() {
    // Tamaño fijo de 5 enteros (20 bytes típicamente)
    int buffer[TAMANO] = {10, 20, 30, 40, 50};

    // No es posible redimensionar 'buffer' en tiempo de ejecución
    printf("Primer elemento: %d\n", buffer[0]);
    return 0;
}
```

b) Estructuras Dinámicas

Las estructuras **dinámicas** asignan y liberan memoria según la necesidad durante la ejecución.

- Su tamaño **puede crecer o contraerse** dinámicamente.
- La memoria se administra en el *Heap* (montículo) mediante funciones de bajo nivel (**malloc**, **free**, etc.).

Ventajas y Desventajas

- ✓ Uso eficiente del espacio en memoria.
- ✓ Flexibilidad para manejar volúmenes de datos variables.
- × Mayor complejidad técnica (riesgo de fugas de memoria o punteros colgados).

Ejemplo de Estructura Dinámica en C

Uso de gestión de memoria dinámica en C para adaptar el tamaño en tiempo de ejecución:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 10;
    // Reservamos memoria dinámicamente en el Heap
    int* arreglo_dinamico = (int*) malloc(n * sizeof(int));

    if (arreglo_dinamico != NULL) {
        arreglo_dinamico[0] = 100;

        // Debe liberarse explícitamente
        free(arreglo_dinamico);
    }
    return 0;
}
```

Resumen: Estáticas vs. Dinámicas

Estructuras Estáticas	Estructuras Dinámicas
Tamaño fijo definido en compilación.	Tamaño variable durante la ejecución.
Memoria en el <i>Stack</i> .	Memoria en el <i>Heap</i> .
Sin overhead de punteros extra.	Requieren punteros para vincular elementos.
Riesgo de falta de espacio o desperdicio.	Riesgo de <i>memory leaks</i> si no se libera.

Cuadro Matriz de Clasificación

Una estructura de datos puede cruzarse entre ambas dimensiones:

Estructura	Lógica	Gestión de Memoria
Arreglo tradicional	Lineal	Estática
Arreglo dinámico (realloc)	Lineal	Dinámica
Lista enlazada	Lineal	Dinámica
Árbol Binario	No Lineal	Dinámica
Grafo con matriz de adyacencia	No Lineal	Estática / Dinámica

Retroalimentación de operaciones con arreglos y estructuras

Repaso General: Arreglos y Estructuras

Dos de las formas más utilizadas para agrupar datos en C son:

Arreglos (Homogéneos)

Colección finita y ordenada de elementos del **mismo tipo de dato**, almacenados en posiciones contiguas de memoria.

Estructuras (Heterogéneas)

Colección de variables (miembros) que pueden ser de **diferentes tipos de datos**, agrupadas bajo un mismo nombre.

Operaciones Fundamentales sobre Arreglos

Las operaciones principales que realizamos sobre arreglos incluyen:

1. **Recorrido / Lectura:** Acceso secuencial a cada elemento.
2. **Inserción:** Agregar un nuevo elemento en una posición específica.
3. **Eliminación:** Remover un elemento y desplazar los restantes.
4. **Búsqueda:** Localizar la posición de un valor (Lineal o Binaria).
5. **Modificación:** Actualizar el valor contenido en un índice.

Operación: Recorrido y Búsqueda Lineal

Demostración de recorrido y búsqueda de un elemento dentro de un arreglo:

```
#include <stdio.h>

int buscar(int arr[], int tam, int clave) {
    for (int i = 0; i < tam; i++) {
        if (arr[i] == clave) {
            return i; // Elemento encontrado en índice i
        }
    }
    return -1; // No encontrado
}
```

Operación: Inserción en Arreglos Fijos

Insertar un elemento en medio de un arreglo requiere **desplazar** los elementos hacia la derecha para hacer espacio.

```
void insertar(int arr[], int *tam, int pos, int valor) {  
    // Desplazamiento a la derecha  
    for (int i = *tam; i > pos; i--) {  
        arr[i] = arr[i - 1];  
    }  
    arr[pos] = valor;  
    (*tam)++; // Incrementamos la cuenta de elementos  
}
```

Paso de Arreglos a Funciones

En C, al pasar un arreglo a una función, este se decae automáticamente a un **puntero al primer elemento**.

```
#include <stdio.h>

// Ambas firmas son equivalentes:
// void modificar(int *arr, int tam)
void modificar(int arr[], int tam) {
    for (int i = 0; i < tam; i++) {
        arr[i] *= 2; // Modifica el arreglo original
    }
}
```

Nota: Es necesario pasar explícitamente el tamaño (**tam**) del arreglo.

Definición de un tipo compuesto usando `struct` y `typedef`:

```
#include <stdio.h>

typedef struct {
    int id;
    char nombre[50];
    float promedio;
} Alumno;

int main() {
    // Inicialización directa
    Alumno a1 = {101, "Carlos Rojas", 9.5};

    printf("ID: %d, Nombre: %s\n", a1.id, a1.nombre);
    return 0;
}
```

Acceso a Campos: Operadores . y ->

Dependiendo de si se trabaja con la variable directa o con un puntero a la estructura:

```
#include <stdio.h>

void imprimir_alumno(const Alumno *a) {
    // Uso del operador flecha -> para accesos mediante puntero
    printf("ID: %d | Promedio: %.2f\n", a->id, a->promedio);
}

int main() {
    Alumno a1 = {102, "Ana Lopez", 9.8};

    // Uso del operador punto . para acceso directo
    a1.promedio = 10.0;

    imprimir_alumno(&a1);
    return 0;
}
```

Arreglos de Estructuras

Combinación de ambas construcciones para manejar listas homogéneas de entidades complejas.

```
#include <stdio.h>

#define MAX 30

typedef struct {
    int codigo;
    float precio;
} Producto;

int main() {
    Producto inventario[MAX]; // Arreglo de 30 estructuras

    inventario[0].codigo = 501;
    inventario[0].precio = 25.50;

    return 0;
}
```

Operación: Búsqueda sobre Arreglo de Estructuras

Iteración sobre un arreglo de estructuras para filtrar datos específicos:

```
int buscar_por_codigo(Producto inv[], int tam, int cod) {  
    for (int i = 0; i < tam; i++) {  
        if (inv[i].codigo == cod) {  
            return i; // Retorna el índice del producto  
        }  
    }  
    return -1;  
}
```

Una estructura puede contener como miembro a otra estructura previamente definida.

```
typedef struct {  
    int dia;  
    int mes;  
    int anio;  
} Fecha;  
  
typedef struct {  
    char titulo[100];  
    Fecha fecha_publicacion; // Estructura anidada  
} Libro;  
  
// Acceso en cadena:  
// libro.fecha_publicacion.anio = 2026;
```

Comparativa de Rendimiento y Memoria

Arreglo Simple	Arreglo de Estructuras
Almacena solo un tipo de dato por lista.	Agrupar múltiples propiedades relacionadas.
Acceso eficiente por desplazamiento.	Puede requerir alineamiento de memoria (<i>padding</i>).
Fácil asignación contigua.	Paso por valor a función puede ser costoso (usar punteros).

Resumen de Buenas Prácticas

1. **Paso por Referencia:** Pasa estructuras grandes a funciones mediante punteros (`const Struct *s`) para evitar copias innecesarias en el `*stack`.
2. **Uso de typedef:** Simplifica la sintaxis al declarar variables de tipo estructura.
3. **Límites de Arreglos:** Controla siempre los índices para prevenir lecturas/escrituras fuera de rango (*Buffer Overflow*).
4. **Paso de Tamaño:** Pasa siempre el número de elementos activos junto con los arreglos a las funciones.